DashFood

FINAL ICT PROJECT

Cafe online ordering system

SE-2622
Artur, Adiyat, Erdaulet
8 November 2026

Executive summary

Small cafes need a consistent way to receive pickup orders without relying on verbal messages or handwritten lists. DashFood provides a responsive web menu, a persistent cart and a checkout connected to a shared PostgreSQL database. Customers submit a pickup name, receive an order reference and follow preparation status. Authenticated staff process the same order, change menu availability and inspect aggregated order data. The final build retains the original project scope and corrects issues identified in the earlier review: hidden mobile navigation, searches containing extra spaces and an edit form that opened outside the viewport. It also protects customer order lookup with a secret tracking key and prevents duplicate orders when a checkout request is retried. Verification covers database operations, customer and staff journeys, mobile layout, accessibility scanning and controlled page-load measurements. The data analysis uses labelled seeded records and synthetic verification orders, so its charts demonstrate a repeatable method rather than real cafe demand. External participants tested the earlier build according to the team, but their task completion times were not recorded. The final assessment therefore separates demonstrated software behaviour from pending human timing, deployment trust and long-term availability evidence.

Review edition. The system and automated evidence are complete for this build. Human timing, individual signatures and deployment/availability evidence remain explicit open items. No separate team name was supplied; DashFood is the project name.

Problem, scope and evidence

Stage 1 identifies a practical ordering problem: customers need to see products and prices before arriving, while staff need one reliable list of pickup orders. A verbal order can lose an item or require repetition. DashFood records the selected products and gives both user groups a shared status.

What the evidence supports

Evidence	Supported conclusion	Limit
Annotated Stage 1, pp2-5	A documented problem, two user groups and three success criteria	No measured local queue baseline
Stage 2, final five pages	Three reproducible interface problems and instrumented page-load results	Not a timed human checkout study
Final browser/API evidence	An order reaches PostgreSQL and the staff screen; status returns to the customer	Controlled synthetic verification
Team clarification	Medet, Nurdaulet and Iskander participated in testing	Task times and individual observations were not supplied

Scope retained

The finished scope covers menu browsing, prices, cart changes, pickup checkout, status tracking, staff order processing and menu control. Insights and a simulated queue rule reuse the same data. Customers pay at pickup. Payroll, delivery tracking, supplier management, reservations, loyalty and online card payments remain outside the project.

Operational outcome

The demonstrated result is an auditable handoff from customer selection to staff preparation. Queue reduction and sales improvement remain hypotheses until a real cafe measures comparable before/after conditions. The final report does not convert a functioning prototype into an unsupported business-impact claim.

Users and functional requirements

Customers use a phone or laptop without registration. Staff sign in as the administrator role. The final build implements one staff role; it does not claim separate cashier, chef or manager permissions. Priorities below preserve Stage 1.

ID	Requirement	Priority	Stage 2	Final / evidence
R1	View menu	Must	Done	Done; menu/browser check
R2	View prices	Must	Done	Done; KZT and API
R3	Add product to cart	Must	Done	Done; repeated add/feedback
R4	Remove cart product	Must	Done	Done; removal/total
R5	Change quantity	Should	Done	Done; 1-50 and reload
R6	Place order	Must	Done	Done; DB transaction, retry check
R7	View status	Should	Done	Done; customer sees Ready
R8	Staff view orders	Must	Done	Done; staff login, guest blocked
R9	Staff change status	Must	Done	Done; legal transitions only
R10	Staff add/edit products	Should	Done	Done; validated menu form
R11	Staff disable products	Must	Inferred done	Function done; wording reconstructed

R11 is cut off in the annotated original. Its visible test says to disable a test product. The implemented disabling behaviour is verified, but the lost source sentence cannot be certified from the supplied files.

Traceability

The repeatable checks are tools/test-api.mjs, tools/test-browser.mjs and tools/final-checks.mjs. Dated JSON and screenshots are stored under output/evidence. The full requirement table and reasons for pending items are in docs/FINAL_REQUIREMENTS.md.

Non-functional acceptance criteria

The annotated Stage 1 notes that R12-R15 lack priority and a meaningful check. The final table explicitly clarifies priority and defines observable checks without changing the original targets.

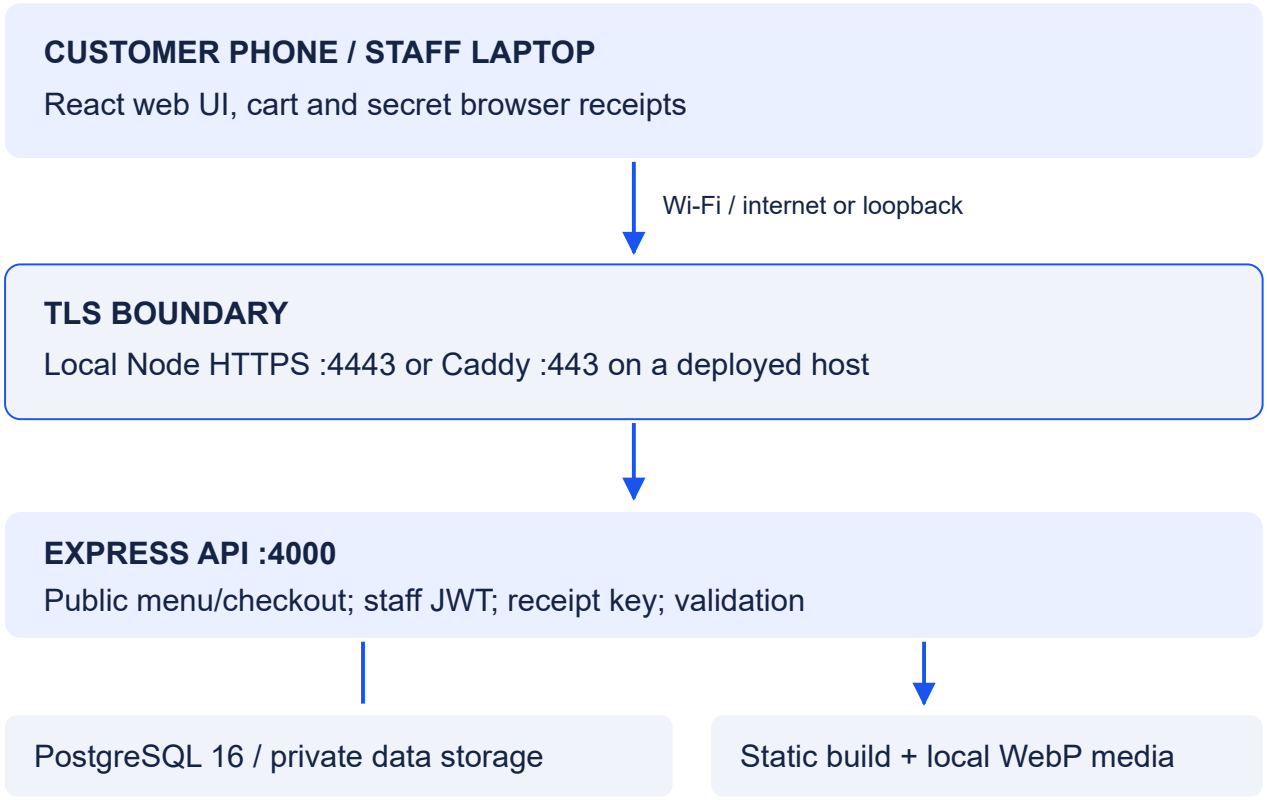
ID / priority	Target and check	Stage 2	Final status
R12 / Must	Menu and cart ready in <2s. Three cold-cache Fast 4G runs at 375x812 DPR1.	1.411s menu; 1.208s cart ready (Nov 1)	Menu 1.015s; cart 0.941s average. Local conditions only.
R13 / Must	HTTPS between customer and server; inspect trusted certificate on deployed host.	Local TLS only	Local TLS verified with self-signed cert. Public trust remains pending.
R14 / Must	No more than five main actions; count a real participant journey.	Four main actions reported	Menu, Add, Cart, name, submit. Human step-count record pending.
R15 / Must	At least 99% availability over an agreed period, including expected probes during downtime.	Unmeasured	Monitoring script supplied. Meaningful observation remains pending.

Three original success criteria

SC1: a customer places an order in less than two minutes. **SC2:** an unavailable product cannot enter a new order. **SC3:** the main ordering interface works at a width of 360 pixels. These come from Stage 1 page 5. The three important requirements on page 8 (R1, R3, R6) are a different list.

Page-load time measures the interface becoming ready. Human task time includes reading, choosing, typing and understanding the confirmation. These metrics cannot substitute for one another. No human time under 120 seconds is claimed.

As-built architecture



The laptop review runs Express and a real PostgreSQL 16 process on loopback. The Docker variant runs the app and database in separate services with a private database network and a named volume. Static React assets and local images come from the app server. No live S3, RDS, EC2, native mobile app or physical IoT device participates in this build.

Boundary	Actual treatment
Public	Menu, product prices and validated guest checkout
Customer-owned	Order read requires a secret receipt token; no token in URL
Staff	Order list, menu writes and analytics require signed admin JWT
Encrypted in transit	Local :4443 TLS; trusted domain deployment remains to verify
At rest	Local PostgreSQL directory / Docker volume; disk encryption is operator configuration

HTTP :4000 remains available for local review and health checks. In public hosting, restrict direct access and serve clients through trusted HTTPS. A configured reverse proxy alone is not proof of a secure public deployment.

Changes from Stage 1 design

Stage 1 plan	Final implementation	Reason
React web + React Native	Responsive React web	A 360px browser journey satisfies the stated mobile scope without a second app.
Gateway / Nginx	Express static server; optional Caddy TLS	One origin simplifies local review; Caddy handles domain certificates.
AWS EC2, RDS and S3	Local PostgreSQL / Docker volume and bundled images	Reproducible course demo; no claimed cloud resources or cloud pricing.
UUID products/categories, user-owned paid orders	Serial IDs, fixed categories, guest pickup orders	Smaller schema for the implemented pickup workflow; payment occurs in person.
Stock quantities and product FK in order items	Availability flag and item name/price snapshots	Scope requires unavailable-product rejection and historical totals, not inventory.
AES-256 storage and universal TLS claim	Actual local storage and explicit optional TLS	Describe verified controls and configuration limits.
General future smart technology	Live order-count wait rule	A transparent simulation reuses existing data; hardware remains future work.

Cost and performance justification

The review uses existing hardware and open-source software. No paid cloud account is required for the local demonstration. Electricity, hardware, internet and future hosting still have costs; no zero-cost production claim is made. The revised final report removes the outdated AWS 12-month free-tier assertion.

A single application process and connection pool keep the demonstration simple. PostgreSQL indexes support order-item joins and chronological queries. Responsive WebP variants reduce image bytes. Current browser measurements assess the actual production build; they do not establish capacity for 10,000 concurrent users.

Data model and transaction design

Table	Important columns	Purpose / relationship
users	id, username, password_hash, role	Unique staff account; bcrypt password hash
products	id, name, category, price, description, active	Whole positive KZT; fixed category check
orders	id, total, status, created_at, customer_name, is_demo	One pickup transaction and its current state
orders: final additions	tracking_hash, request_key, request_hash	Customer read authorization and retry deduplication
order_items	id, order_id, name, price, qty	Many items per order; FK to orders with cascading delete
seed_batches	name, created_at	Marks one repeatable synthetic seed batch

Checkout transaction

The API validates the pickup name and unique product lines. It locks one checkout request key using a transaction-level advisory lock, then checks for a prior order. Repeating the same payload returns the saved receipt; changing that payload with the same key returns a conflict. The server reads active product rows with a shared lock, obtains their names/prices and calculates the total.

It inserts orders and order_items, commits them together, then returns the receipt. Failures roll back the transaction. Row locks prevent a concurrent menu edit from changing price or availability midway through this write. A unique index is an additional guard on request_key.

Historical correctness

Each order item retains the purchased name, price and quantity. Editing a menu price later does not rewrite an old receipt. Integer KZT avoids floating-point currency rounding in this scope. Quantity is limited to 50 per product and the total to 10,000,000 KZT. Parameterized queries treat customer text as data.

The schema is additive for existing installations. Legacy orders without a tracking hash remain staff-visible but cannot be opened publicly by guessing their IDs. No old customer tracking credentials are fabricated.

Security, privacy and failure handling

Risk	Implemented control	Remaining limit
Staff impersonation	bcrypt hashes, HS256 JWT verification, eight-hour expiry, login rate limit	No MFA or managed account administration
Guessing another order	Secret token returned at checkout; only SHA-256 token hash stored; constant-time comparison	Browser storage must be retained and protected
Price manipulation	Prices read from DB; customer-supplied total ignored	No inventory reservation or online payment
Double checkout after lost response	Persistent checkout key and serialized server retries	Different keys represent different orders
Malformed input or SQL injection	JSON size cap, type/range checks, parameterized SQL, valid identifiers	No claim of a complete penetration test
Invalid staff actions	Atomic state transition guards; terminal statuses cannot jump backwards	One admin role, no detailed staff audit log

Personal data and location

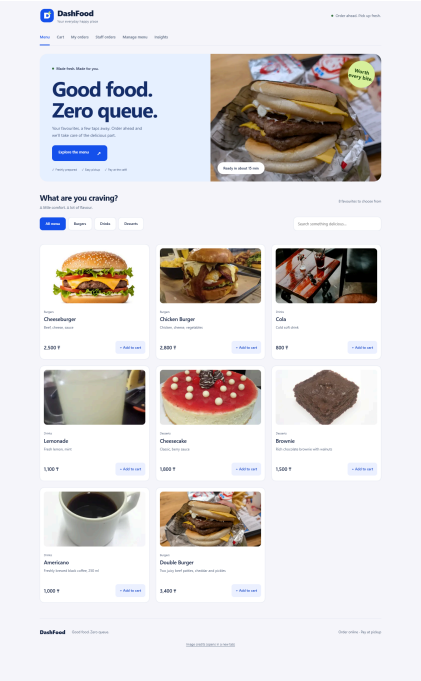
Pickup names are stored in the PostgreSQL orders table and shown only to authenticated staff. Customer order read omits the name. The application collects no phone, address or payment-card details. In this review, DB files reside on the laptop under .local; in Docker they reside in a named volume on the deployment host. No cloud location is assumed.

Before real use, the operator needs a documented hosting location, retention/deletion policy, encrypted backups and an appropriate review of Kazakhstan personal-data obligations. Local synthetic testing does not establish legal compliance. The final build does not claim automatic AES-256 encryption of database files.

Visible recovery

Errors keep the cart intact, unavailable items can be removed, and retries preserve the checkout key. A missing receipt no longer prevents other customer orders from loading. Expired staff sessions return to sign-in. Non-API routes serve the app, while unknown API routes return JSON 404.

Customer interface and multimedia



Final production menu captured during the database-backed browser journey.

The menu exposes names, descriptions, categories and KZT prices before checkout. Search ignores surrounding spaces. Repeated add actions announce visible feedback and update cart quantity/total. Cart controls support removal and a 1-50 quantity range. Checkout asks for a pickup name and explains payment at the cafe.

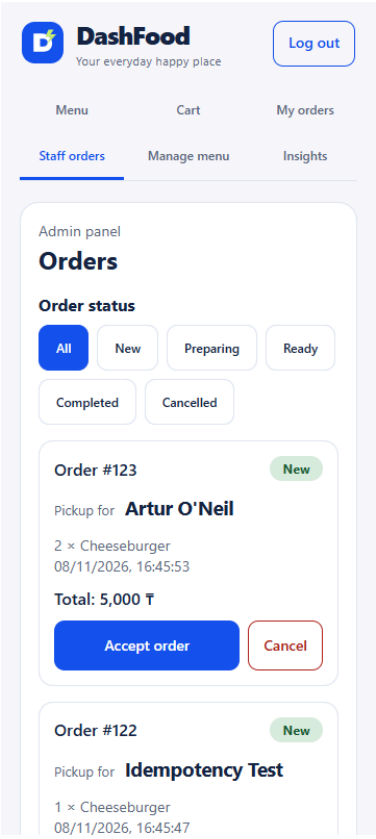
Media and accessibility decisions

Images use local WebP variants, explicit dimensions, srcSet/sizes and descriptive alternative text. The hero image loads eagerly; non-priority cards can load lazily. Unknown products receive a stable fallback. Credits remain in the linked local media credits page. Example photography is not presented as actual cafe stock.

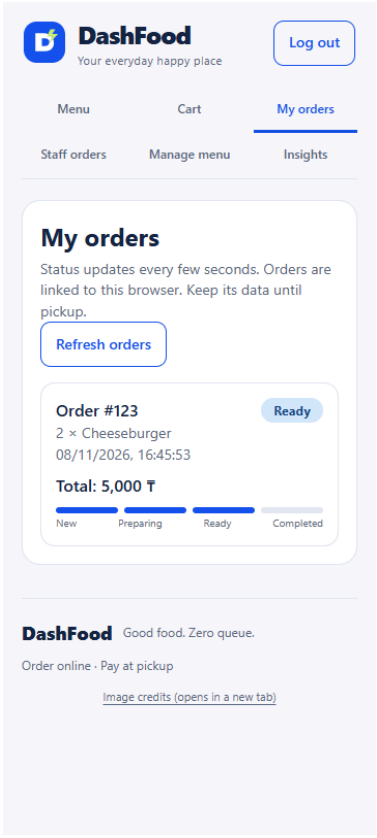
Main buttons have minimum heights of 44 or 48 CSS pixels. Keyboard focus is visible. Reduced-motion preference disables animations and smooth scrolling. Text appears on opaque surfaces rather than photographs. Brand text #1854ed on white has a calculated contrast of 5.95:1; secondary text #5d6a82 on white is 5.46:1. Ratios use the WCAG sRGB luminance formula.

Automated accessibility scanning covers six screens at 360px. It cannot certify every screen-reader experience. User-controlled backup video is a separate presentation artifact; the application plays no automatic audio.

Staff journey and customer handoff



Authenticated staff receive the saved pickup order.



Customer reads Ready after the staff action.

Staff sign in to see the same PostgreSQL order created by the customer. The required workflow is New, Preparing, Ready and Completed. Cancellation is allowed from New or Preparing. Attempts to skip stages or update an already changed order are rejected. Busy controls reduce accidental repeated actions.

Manage menu supports creating a product, editing name/description/category/whole KZT price and toggling availability. An unavailable product disappears from public menu responses and cannot be purchased from a stale cart. Editing the last product scrolls to the form and focuses its Name field; Cancel restores the original Edit button.

Two user groups in one demonstration

The live presentation must show both journeys: customer checkout and tracking, then staff processing and menu control. Showing only the home page does not demonstrate the shared database. Use the created order number to connect the browser action, the staff card and the SQL query.

Final testing and Stage 2 comparison

Success criterion	Target	Stage 2	Final / met?
SC1: place an order	<120s human task	Wrong task labels; no human duration	Automated journey passes. External timing remains unmeasured. Not yet confirmed.
SC2: unavailable product	No unavailable order accepted	API rejection reported	Final API rejects it; public menu excludes it. Automated pass, human record pending.
SC3: mobile interface	360px full journey	375px evidence on last slides	360px audited plus 320/375/768/1440 layout checks. Automated pass, human record pending.

Actual final verification

53 API assertions passed. They include staff authorization, invalid data, DB pricing, transaction results, workflow guards, disabled products, secret tracking and five concurrent retry requests creating exactly one order. The real browser journey produced no uncaught page errors.

6 accessibility scans cover menu, cart, login, staff orders, product editor and Insights at 360px. The final run detected 0 WCAG A/AA violations in that automated scope. These results are dated 2026-11-08 and use 156.0.8078.80.

External participants

The team confirms testing with Medet, Nurdaulet and Iskander but did not measure checkout times. The old document also says B1 remains incomplete. The final table preserves that gap rather than estimating a duration from the recording or script. docs/FINAL_TESTING.md provides the short retest and per-participant fields.

Record real device/browser, 360px viewport, completion seconds, help/errors, negative-product result and main action count. Retain failed observations as well as successful ones. Update the final report only after collecting these values.

Resolved review findings and performance

Finding from Stage 2	Final correction	Repeatable result
F01: right-hand routes hidden on mobile	All six navigation actions appear in a grid at narrow widths	All nav button bounds fit at 320/360/375/768/1440px
F02: padded search returns no result	Trim before matching, whitespace-only query is empty search	Padded Cheeseburger finds one card; spaces restore list
F03: form opens above the viewport	Immediate scroll/focus to Name; restore focus on Cancel	Last product form visible/focused at 360px; return focus verified

Production-build measurement

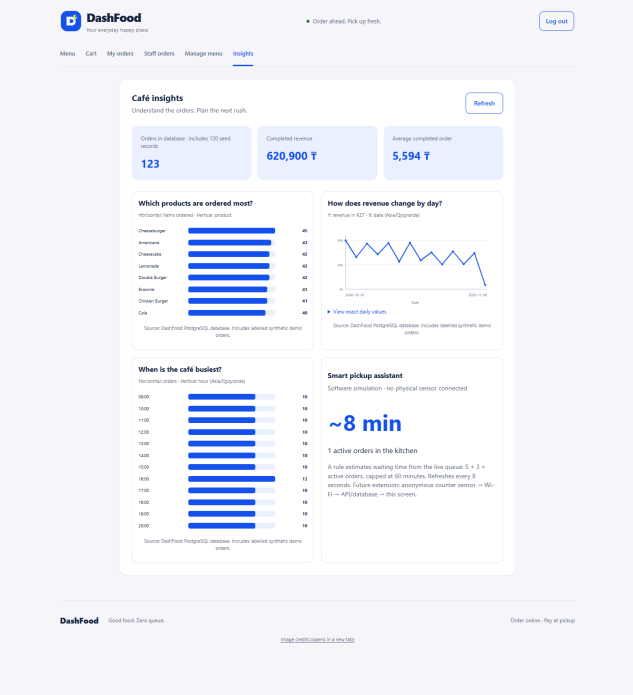
Screen	Ready run 1	Run 2	Run 3	Mean ready	Mean load
menu	1.027s	1.000s	1.019s	1.015s	0.594s
cart	0.922s	0.961s	0.940s	0.941s	0.578s

Conditions: fresh browser contexts, cache disabled, 375x812 CSS pixels, DPR1, 1,012,500 bytes/s down, 168,750 bytes/s up and 165ms latency through Chrome DevTools Protocol. Ready means product cards or cart quantity controls become observable. It is not human task completion. The JSON retains each run and browser version.

Stage 2 reported 1.411s menu and 1.208s cart readiness on November 1. The final run uses a changed build and a different session; the comparison is descriptive, not a controlled causal performance-improvement claim. App screenshots are not screenshots of the Network panel. Manual Network captures remain a separate evidence task if required by the instructor.

R12 is assessed only under the recorded local test conditions. Internet distance, host resources, caching and device speed can change the result.

Data analysis and interpretation



Actual Insights capture. The analysis snapshot is synthetic course demonstration data.

Question	Query / unit	Current captured result
Which items are ordered most?	SUM(item.qty), excluding Cancelled; units	Leader: Cheeseburger (45)
How does revenue vary by day?	SUM(order.total) for Completed, local date; KZT	Completed total 620,900 KZT
When are requests busiest?	COUNT(*) grouped by local hour; orders	Peak 16:00

The snapshot contains 123 orders, of which 120 are flagged seed records. Remaining records in this assessment were also created by verification, not paying customers. Average completed order: 5,594 KZT. Capture time: 2026-11-08T11:46:10.864Z.

These queries test an information-system method: join order items to orders, select a status population, aggregate with clear units and display the result. The sample does not establish real popularity, staffing demand or sales. A real operational study would separate training data, collect representative periods and account for cancellations and opening hours.

Smart extension and its limits

Implemented simulation

Insights calculates an estimated waiting time using **min(60, 5 + 3 x active orders)** minutes. Active means New or Preparing. The screen refreshes every eight seconds and explicitly states that no physical sensor is connected. Staff actions change the queue and therefore the estimate.

Active orders	Calculated estimate	Meaning
0	5 minutes	Fixed baseline
3	14 minutes	5 + 3 x 3
20	60 minutes	Uncapped 65 reaches the 60-minute limit

The formula is a transparent software rule. It has not been calibrated against kitchen preparation times and must not be presented as an accurate promise to customers. The menu hero should avoid treating a fixed promotional estimate as a measured live value.

Future anonymous occupancy counter

A possible extension uses a time-of-flight pickup counter, Wi-Fi with per-device credentials, authenticated HTTPS ingestion and a sensor_readings table. The staff screen would combine aggregate occupancy with the current queue. No camera, face recognition or phone identifiers are needed for this proposed design.

At one reading per ten seconds, a device sends 8,640 readings per day. Assuming 100 bytes of JSON per reading gives about 0.864 MB of raw JSON, excluding network overhead, retries and storage indexes. A future retention proposal is seven days for raw data and 30 days for minute aggregates. These are design assumptions, not deployed collection or retention policies.

Failure modes

Connectivity loss needs a stale-data indicator; duplicate sequence numbers need deduplication; impossible counts need rejection. New presence patterns may still affect privacy even without names. No sensor-ingestion route, hardware test or production accuracy is claimed.

Deployment, reproducibility and recovery

One local review command

Install dependencies with npm run setup, then run npm run review. It compiles the app, starts a persistent local PostgreSQL 16 instance, applies additive schema changes, inserts the seed batch once and starts HTTP/TLS endpoints. StartPreview.cmd provides the Windows entry point. The README lists the guest and staff role credentials and where generated passwords live.

Alternative Docker path

Open Docker Desktop and run setup.ps1 on Windows or bash setup.sh on macOS/Linux. App and database health checks verify the connection. A named volume persists data across ordinary stops. The optional Caddy profile accepts an owned domain for HTTPS. Docker was not available in the assessment session, so the final verified execution uses native local PostgreSQL rather than claiming a fresh container deployment.

Backup and restore exercise

For Docker, an operator can export with docker compose exec -T db pg_dump -U cafe -d cafe and save the output securely. Rehearse restoration into a separate test database before relying on a backup. A backup is sensitive because it contains pickup names. Do not publish it or store it in Git. Ordinary restart must preserve existing orders.

Availability measurement

tools/monitor.mjs records timestamped health responses, latency, expected sample count and the observed window. A meaningful R15 assessment needs a declared duration and inclusion of missed probes during downtime. Restart policies and a successful launch demonstrate recovery mechanisms; they do not by themselves establish 99% uptime.

Repository evidence

Existing history remains intact. The final work stays on a local review branch until approved. GitHub Actions is configured to run build, API and browser/accessibility checks using PostgreSQL 16. Remote execution is pending push; no green remote checks or new participant commits are invented.

Limitations and final action register

Open item	Required evidence / next action	Owner for confirmation
Human checkout timing	Repeat <120s task with three external users and record actual times	Erdaulet + participants
360px/disabled product human observations	Record outcomes in FINAL_TESTING.md, including any help or failure	Testing facilitator
R11 exact wording	Obtain the uncropped original or confirm wording with instructor	Whole team
Trusted HTTPS	Show valid deployed certificate and close direct public HTTP exposure	Deployment reviewer
R15 >=99% availability	Declare observation period; collect and interpret health probes	Deployment reviewer
B2 contribution statements	Each member writes/verifies 100-120 words with files, problem, reflection, signature	Artur / Adiyat / Erdaulet
Individual Git history	Review each own change, commit with own account, retain honest history	Each member
Final presentation/LMS	Rehearse 8-10 minutes; each member speaks; upload once before course deadline	Whole team

Future engineering work

Before a real cafe launch, add explicit data-retention controls, backup/restore verification, stronger operator account management and monitoring. A higher-load deployment would need bounded/paginated staff order queries, rate limits on public ordering, measured load tests and a capacity plan. The current aggregate queries and unbounded staff list are likely scaling pressure points.

No inventory, delivery, card payment or calibrated kitchen forecast is supplied. These omissions retain the original pickup scope. The most useful immediate work is completing evidence and practicing the existing system rather than adding another feature before defense.

A functioning build and polished report cannot substitute for measurements, personal signatures or a live explanation. Pending criteria remain visible in the final requirement and test tables.

Individual contributions and defense

The team describes development as shared work, so assigning entire modules to individuals after the fact would be misleading. Stage 1 roles identify planned responsibilities, while actual authorship must be checked against files and commits. The existing history includes arqode and adiat-syr work; no retrospective account attribution is created.

Member	Specific final review focus	Personal evidence to add
Artur	Customer interface, mobile layout and menu/cart usability	Own file/commit, solved problem, reflection and signature
Adiyat	API, data model, query/transaction behaviour and deployment boundaries	Own file/commit, solved problem, reflection and signature
Erdaulet	Correct success criteria, instructor feedback, evaluation and defense package	Review final docs, repeat checks, then own final commit

Statement format: 100-120 words per member

Each member should state exactly what they personally built or verified, link to the corresponding real files/commits, explain one problem and how they solved it, and name one improvement for next time. Sign only after reviewing that statement. docs/CONTRIBUTIONS.md provides a worksheet. Signed statements are pending; this page does not impersonate them.

Erdaulet commit preparation

The requested identity is Erdaulet <q7998378@gmail.com>, GitHub q7998378-svg. After personal review, the final evaluation changes can be committed with that identity. The account email must be associated with GitHub for profile attribution. A final commit proves the current contribution, not all previous team development.

Individual defense readiness

Each member needs to explain both user journeys and the full browser/API/database path. Practice adding a validated form field, modifying a SQL aggregate and finding a specific commit. The Russian defense guide maps exact files and provides a nine-minute shared presentation plan and 3-5-minute question practice.

References and submission checklist

Reference	Use in this report
ICT_Final (1).pdf, supplied course brief	Final A-D criteria, 12-20 pages, 8-10 minute presentation, personal defense
midterm_ANNOTATED.pdf, supplied Stage 1	Original success criteria p5, requirements p7, instructor annotations
ICT_EndtermAdiyatEraArtur.pdf, supplied Stage 2	Prior results and internal findings F01-F03; final five pages
github.com/Arqode/CafeSystem	Original project and preserved history; baseline 7a39fcf
PostgreSQL 16: Transactions; Explicit Locking	Transaction/row/advisory lock concepts used in checkout
Express: Production Security	TLS, input handling and dependency hygiene
Vite: Getting Started	Production client build and supported runtime
Embedded PostgreSQL project	Local PostgreSQL distribution/runtime setup
web/public/media/credits.html and provenance.json	Project image sources and credits
output/evidence/*.json and screenshots, 8 Nov 2026	Current API, UI, accessibility, performance and dataset evidence

Technical references accessed 8 November 2026. Course textbooks remain defense preparation material.

Before LMS submission

Collect human timings/observations, confirm R11, complete and sign individual statements, verify each own commit, supply trusted TLS and availability evidence, rehearse all speakers, and upload one team package under the required name. The course deadline must be confirmed from the syllabus; no date is invented.

Report name: SE2622_DashFood_ICT_FinalProject.pdf. DashFood fills the project identifier because the group has no separate team name. This review edition records the remaining evidence honestly.